

Rust Concepts Cheatsheet

A readable guide to the common Rust terms and patterns used in a crash-proof file pipeline project.

Use this when reading code that watches folders, processes files safely, tracks progress, and resumes after crashes.

Core Terms

Term	What it means	Example
fn	Defines a function.	<pre>fn process(path: &Path) -> Result<> { ... }</pre>
let	Creates an immutable variable.	<pre>let path = PathBuf::from("input.txt");</pre>
let mut	Creates a mutable variable that can change.	<pre>let mut count = 0;</pre>
String	Owned text data.	<pre>let s = String::from("hello");</pre>
&str;	Borrowed string slice, usually for read-only text.	<pre>let name: &str = "rust";</pre>
struct	Groups related data into one custom type.	<pre>struct Job { input: PathBuf, state: Stage }</pre>
enum	Represents one of several possible states or variants.	<pre>enum Stage { Queued, Hashing, Done, Failed }</pre>
impl	Adds methods to a struct or enum.	<pre>impl Job { fn new(...) -> Self { ... } }</pre>

Ownership and Borrowing

Ownership means every value has one clear owner. When ownership moves, the old variable can no longer use that value.

```
let s = String::from("hello");
let t = s; // ownership moved to t
```

Borrowing lets code use a value without taking ownership of it. A borrowed reference starts with **&**.

```
fn process(path: &Path) {
    println!("{:?}", path);
}
```

In pipeline code, owned types like **PathBuf** store paths long-term, while borrowed types like **&Path** let functions read paths cheaply.

Error Handling

Result<T, E> is Rust's success-or-failure type. **Ok(value)** means success. **Err(error)** means failure.

```
fn read_file(path: &Path) -> Result<String> {  
    let text = std::fs::read_to_string(path)?;  
    Ok(text)  
}
```

The **?** operator means: if this fails, return the error now; otherwise keep going with the successful value.

Option<T> is for data that may or may not exist yet. Use **Some(value)** when present and **None** when absent.

```
struct Job {  
    hash: Option<String>,  
}
```

State Machines with Enums

Enums are a great fit for file pipelines because jobs move through a fixed set of stages.

```
enum Stage {  
    Queued,  
    Compressing,  
    Hashing,  
    Moving,  
    Done,  
    Failed(String),  
}
```

This is safer than a bunch of booleans because the state is explicit and invalid combinations are harder to create.

Use **match** to handle every possible enum variant.

```
match job.state {  
    Stage::Queued => println!("waiting"),  
    Stage::Hashing => println!("hashing"),  
    Stage::Done => println!("finished"),  
    Stage::Failed(msg) => println!("failed: {}", msg),  
    _ => {}  
}
```

Filesystem Types You Will See Often

Item	Purpose	Example
Path	Borrowed filesystem path. Good for function parameters.	<pre>fn open(path: &Path)</pre>
PathBuf	Owned path. Good for storing and building paths.	<pre>let p = PathBuf::from("data/in.txt");</pre>
std::fs::read	Reads file bytes.	<pre>std::fs::read(path)?;</pre>
std::fs::write	Writes bytes to a file.	<pre>std::fs::write(path, data)?;</pre>

Item	Purpose	Example
<code>std::fs::rename</code>	Moves or renames a file.	<code>std::fs::rename(tmp, final_path)?;</code>
<code>create_dir_all</code>	Creates folders, including missing parents.	<code>std::fs::create_dir_all(out_dir)?;</code>

Why Rust Feels Good for Reliable Pipelines

- **Ownership** helps prevent accidental misuse of file data.
- **Result** makes failures explicit instead of hidden.
- **Enums** make job states clear and safe.
- **Atomic rename patterns** help avoid half-written final files.
- **Structs + impl blocks** keep related data and behavior together.

A common safe pattern is to write to a temporary file first, then rename it into place only after the write is complete.

```
let tmp = out_dir.join("file.tmp");
let final_path = out_dir.join("file.gz");

std::fs::write(&tmp, compressed_bytes)?;
std::fs::rename(&tmp, &final_path)?;
```

Reading Rust Quickly

fn run(path: &Path;) -> Result<()> - A function that borrows a path and either succeeds or returns an error.

impl Job - Methods attached to the Job type.

Some(value) - A value exists.

None - No value exists yet.

? - Return the error now if this step failed.

&self; - Borrow this object for read-only access.

&mut; self - Borrow this object for editable access.

Best order to learn these concepts

Start with: **ownership**, **borrowing**, **Result**, **Option**, **struct**, **enum**, **match**, and **Path vs PathBuf**.